Design Document: Pong Game Kernel System

Chawin Luangtumrongjareon

1. Overview

This document describes the design of a Rust-based kernel system that implements a Pong game on x86_64 architecture. The system uses the bootloader API to initialize hardware, manages interrupts for timer and keyboard input, allocates memory, handles graphics rendering via a framebuffer, and maintains game state. It interacts with the CPU and hardware through low-level mechanisms like the Global Descriptor Table (GDT), Interrupt Descriptor Table (IDT), and Local APIC (LAPIC).

2. System Architecture

2.1 High-Level Components

The system consists of the following major components:

- **Kernel Core**: Manages initialization, game logic, and hardware interaction (main.rs).
- **Interrupt Handler**: Processes hardware interrupts (timer and keyboard) via the IDT and LAPIC (interrupts.rs).
- **Memory Manager**: Allocates and manages physical and virtual memory (allocator.rs, frame_allocator.rs).
- **Graphics Renderer**: Draws the Pong game on a framebuffer (screen.rs).
- **Global Descriptor Table (GDT)**: Sets up segmentation and task switching (gdt.rs).

2.2 Interaction with Kernels

The system does not run on a traditional operating system kernel but acts as a bare-metal kernel itself, directly interacting with x86_64 hardware. It uses:

- **System Calls and Interrupts**: Handles interrupts (e.g., timer, keyboard) via the IDT and LAPIC, bypassing a higher-level kernel.
- **Memory Mapping**: Maps physical memory to virtual addresses using the page table and frame allocator.
- **I/O Operations**: Communicates with hardware via port I/O (e.g., keyboard) and memory-mapped I/O (e.g., LAPIC, framebuffer).

The kernel design is minimalist, focusing on real-time game updates and low-level hardware control.

3. Kernel Design

3.1 Kernel Structure

The kernel is structured as a single-threaded, event-driven system:

- **Entry Point**: kernel_main in main.rs initializes hardware, sets up memory, and starts the game.
- **Interrupt-Driven**: Uses the IDT to handle timer ticks (for game updates) and keyboard inputs (for player control).
- **APIC Management**: Configures Local APIC for interrupt handling and I/O APIC for routing interrupts.

3.2 Kernel Interaction

This system is a game kernel. It interacts with hardware directly:

- **Interrupt Handling**: The interrupts.rs module sets up handlers for timer and keyboard interrupts, which call game logic functions (tick and key in main.rs).
- **Hardware Abstraction**: Uses libraries like x86_64 and bootloader_api to abstract CPU registers, paging, and boot information.

4. Memory Allocator

4.1 Design

The memory allocator is a simple bump allocator (allocator.rs):

- **Allocation Strategy**: Allocates memory linearly from a fixed heap starting at HEAP_START with a size of 100 KiB (HEAP_SIZE).
- **No Deallocation**: The dealloc function is implemented but only logs the call, as the bump allocator does not support freeing memory (suitable for a simple kernel with static memory needs).

4.2 Key Features

- **Static Heap**: Initialized with a physical memory offset from the bootloader, ensuring the heap starts at a usable memory region.
- **Error Handling**: Panics if out of memory, which is acceptable for a simple system but could be enhanced for robustness. Can run out of memory if there have been too many rounds played up to the current memory.

5. Input Controller

5.1 Functionality

The input controller is part of interrupts.rs and handles:

- **Keyboard Input**: Uses the pc_keyboard crate to decode scancodes from the keyboard port (0x60). It processes keys like 'W', 'S' (left paddle), and Arrow Up/Down (right paddle) in the keyboard_interrupt_handler.
- **Interrupt-Driven**: Keyboard interrupts are routed via the LAPIC and IDT, triggering the key function in main.rs to update paddle positions.

5.2 Workflow

1. Keyboard interrupt happens, reading scancode from port 0x60.
2. The keyboard_interrupt_handler decodes the scancode into a key event using pc_keyboard.
3. If valid, it calls the registered handler (handle_keyboard) via HANDLERS, which invokes the key function to adjust paddle positions.

6. State Management

6.1 Design

The state manager maintains the Pong game state using global static variables and atomic integers:

- **Game Objects**: PADDLE_LEFT, PADDLE_RIGHT, BALL_X, BALL_Y, BALL_SPEED_X, BALL_SPEED_Y track positions and velocities.
- **Scores**: LEFT_SCORE and RIGHT_SCORE ensure thread-safe updates during interrupts.
- **Game State**: GAME_STATE tracks whether the game is ongoing (0) or ended (1).

6.2 Storage

- **In-Memory**: All state is stored in global variables, accessed directly by interrupt handlers and game logic.
- **Synchronization**: Uses AtomicI32 for score and game state to handle concurrent updates from interrupts safely. Unsafe statics for positions and speeds are accessed carefully to avoid race conditions, assuming single-threaded execution.

6.3 Synchronization

- Interrupt handlers (timer_interrupt_handler, keyboard_interrupt_handler) use mutex-protected HANDLERS to ensure only one handler is active at a time.
- Atomic operations prevent data races on scores and game state.

7. Additional Features

7.1 Graphics Rendering

- **Framebuffer**: screen.rs uses the bootloader's framebuffer to render the Pong game, including paddles, ball, midline, and scores.
- **Drawing Functions**: Methods like draw_pong_pad, draw_ball, draw_mid_line, and draw_zero/draw_one/draw_two/draw_three render game elements in white on a black background.

7.2 Game Logic

- **Tick-Based Updates**: The tick function updates the ball position every timer interrupt, checks collisions with paddles and walls, and updates scores.
- **Win Condition**: The game ends when a player reaches 3 points, displaying a win message and allowing restart with 'r'.

7.3 Interrupt Handling

- Configures LAPIC and I/O APIC for timer (0x20) and keyboard interrupts, ensuring real-time response.

8. Performance Optimization

- **Minimal Overhead**: Uses a bump allocator and simple interrupt handling to keep performance high.
- **Frame Buffer Efficiency**: Only redraws changed elements (e.g., ball, paddles) to minimize framebuffer updates.

9. Error Handling and Recovery

- **Panics**: The system panics on critical errors like out-of-memory or page faults, which is typical for a bare-metal kernel.
- **Interrupt Safety**: Ensures interrupt handlers end with end_interrupt to clear the LAPIC EOI register, preventing interrupt storms.

Design Document: Pong Game Kernel System